

Wirtualny CFO – Dokumentacja techniczna

Projekt: KSEF | Wersja dokumentu: 1.1 | Data: 2026-06-30

Zakres: backend (FastAPI, PostgreSQL, ETL, AI, KSeF, analityka) + minimalny opis konsumpcji API przez frontend.

Repozytorium: c:\Users\pawel.kosinski\source\repos\KSEF

Spis treści

1. Cel systemu i architektura
2. Struktura repozytorium
3. Uruchomienie i konfiguracja
4. Baza danych: modele, migracje, RLS
5. Uwierzytelnianie i multi-tenancy
6. Warstwa HTTP API
7. Potok ETL (XML → AI → PostgreSQL)
8. Parser FA(3) XML
9. Moduł kategoryzacji AI (Ollama)
10. Integracja KSeF API 2.0
11. Analityka opisowa (StatisticsService)
12. Konsumpcja API przez frontend
13. Skrypty deweloperskie
14. Testy automatyczne
15. Diagramy przepływu danych
16. Mapowanie FA(3) → baza danych
17. Luki implementacyjne i roadmap

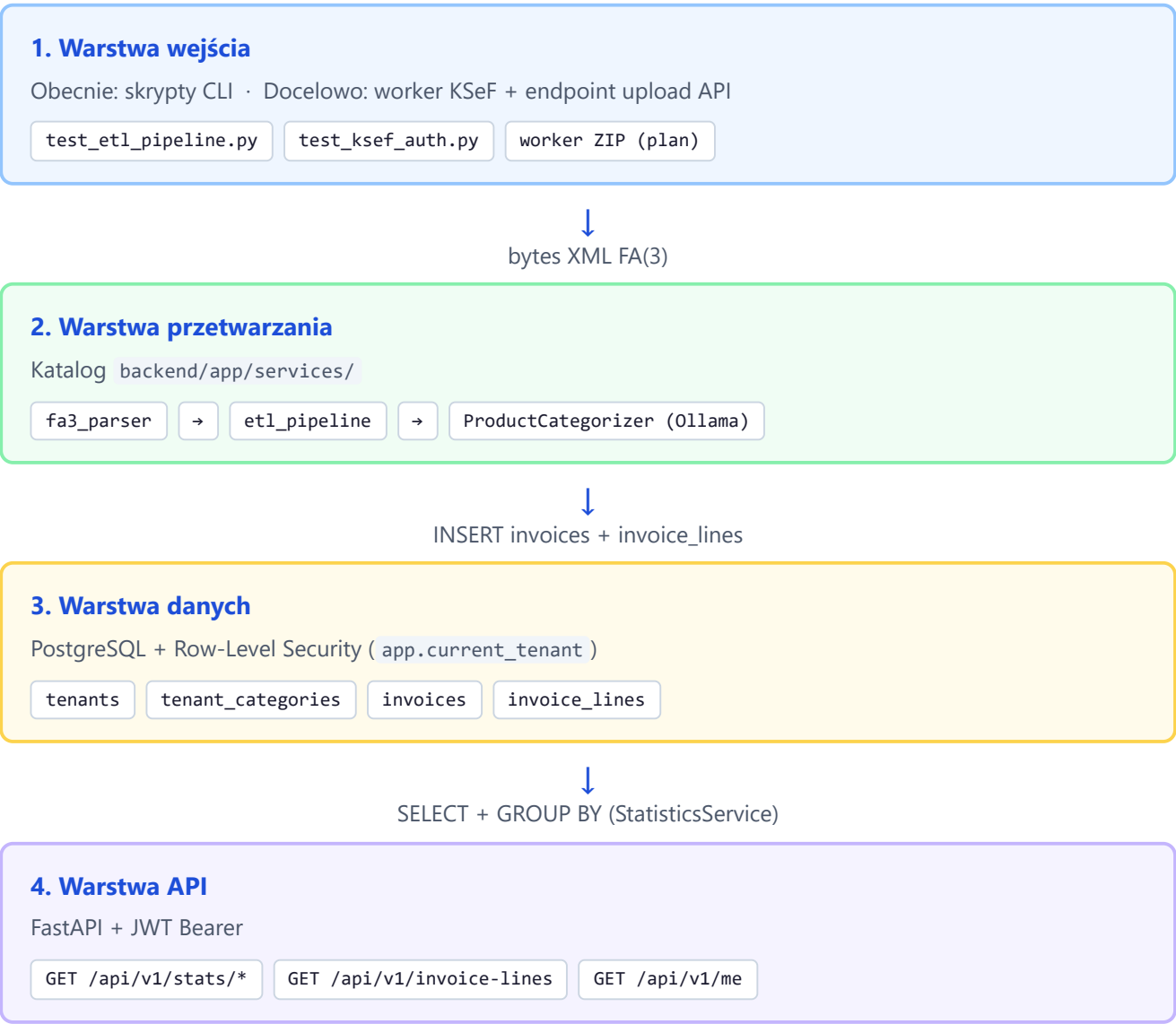
1. Cel systemu i architektura

Wirtualny CFO to MVP platformy B2B dla polskich MŚP. System pobiera faktury kosztowe ze schematu **FA(3)** (Krajowy System e-Faktur), parsuje je, kategoryzuje pozycje towarowe/usługowe za pomocą lokalnego modelu językowego (Ollama), zapisuje wyniki w **PostgreSQL** z izolacją danych per firma (tenant), a następnie udostępnia zagregowane statystyki przez REST API.

1.1. Stos technologiczny (backend)

Warstwa	Technologia	Plik konfiguracyjny
HTTP API	FastAPI 0.115+, Uvicorn	backend/app/main.py
ORM / DB	SQLAlchemy 2.x async, asyncpg	backend/app/database/
Migracje	Alembic	backend/alembic/
Auth (dev)	JWT HS256 (python-jose)	backend/app/dependencies/tenant.py
XML	lxml (parser obronny)	backend/app/services/xml/fa3_parser.py
AI	Ollama AsyncClient, model qwen2.5:7b	backend/app/services/ai/
KSeF	httpx async, RSA-OAEP SHA-256	backend/app/services/ksef/
Baza	PostgreSQL 16 (Docker)	docker-compose.yml

1.2. Architektura warstwowa



↓
JSON

5. Konsument

Frontend Next.js — `apiFetch()` + wykresy Recharts (bez logiki biznesowej)

Kluczowa zasada projektu: **do modelu AI trafia wyłącznie pole P_7 (nazwa towaru/usługi)**. NIP, kwoty, daty i metadane faktury nigdy nie są wysyłane do Ollamy — chroni to prywatność i zapobiega „wyciekowi” danych finansowych do promptu.

2. Struktura repozytorium

```
KSEF/
├── docker-compose.yml          # PostgreSQL 16 (kontener vcfo-postgres)
├── backend/
│   ├── alembic.ini
│   ├── requirements.txt
│   ├── .env.example
│   ├── alembic/
│   │   ├── env.py              # Runner migracji (async)
│   │   └── versions/
│   │       ├── 001_initial_schema_with_qls.py
│   │       ├── 002_tenant_categories.py
│   │       └── 003_force_qls.py
│   ├── app/
│   │   ├── main.py             # FastAPI app, CORS, endpointy główne
│   │   ├── config.py           # pydantic-settings
│   │   ├── api/v1/
│   │   │   ├── router.py       # Agregator routerów v1
│   │   │   └── endpoints/stats.py # Endpointy analityki
│   │   ├── database/
│   │   │   ├── models.py       # SQLAlchemy ORM
│   │   │   └── session.py      # Engine, sesje, set_tenant_context
│   │   ├── dependencies/tenant.py # JWT, TenantContext, get_qls_session
│   │   ├── middleware/tenant_qls.py
│   │   ├── schemas/stats.py    # Pydantic response models
│   │   └── services/
│   │       ├── etl_pipeline.py
│   │       ├── tenant_categories.py
│   │       ├── ai/              # categorizer, schemas, exceptions
│   │       ├── analytics/statistics.py
│   │       ├── ksef/            # client, auth, crypto, models
│   │       └── xml/             # fa3_parser, models
│   ├── scripts/                # Skrypty integracyjne (5 plików)
│   └── tests/                  # pytest (6 modułów + fixture XML)
└── frontend/                  # Next.js - konsument API (poza zakresem UI)
```

3. Uruchomienie i konfiguracja

3.1. Kolejność uruchomienia pełnego stacku

1. `docker compose up -d` — PostgreSQL na porcie 5432
2. `cd backend && .venv\Scripts\alembic upgrade head` — migracje
3. Ollama z modelem `qwen2.5:7b` na `http://localhost:11434`
4. `.venv\Scripts\uvicorn app.main:app --reload --port 8000`
5. `.venv\Scripts\python scripts\test_etl_pipeline.py` — załadowanie danych testowych
6. `.venv\Scripts\python scripts\create_dev_token.py --nlp 9998887776` — token JWT
7. Frontend: `NEXT_PUBLIC_DEV_TOKEN` w `.env.local`, `npm run dev`

Ważne (Windows): Zawsze używaj `backend\.venv\Scripts\python.exe`, nie systemowego `python`. Systemowy interpreter nie ma zainstalowanych zależności (sqlalchemy, fastapi itd.).

3.2. Zmienne środowiskowe (backend/.env)

Zmienna	Domyślnie	Opis
<code>DATABASE_URL</code>	<code>postgresql+asyncpg://vcfo:vcfo_secret@localhost:5432/wirtualny_cfo</code>	Connection string async
<code>JWT_SECRET_KEY</code>	<code>change-me-in-production</code>	Klucz HS256 — musi być zgodny z tokenem frontendu
<code>JWT_ALGORITHM</code>	<code>HS256</code>	Algorytm JWT
<code>DEBUG</code>	<code>false</code>	<code>true</code> → echo SQL w logach
<code>KSEF_BASE_URL</code>	<code>https://api-test.ksef.mf.gov.pl/api/v2</code>	Środowisko testowe KSeF
<code>KSEF_NIP</code>	<code>(puste)</code>	NIP kontekstu uwierzytelniania KSeF
<code>KSEF_TOKEN</code>	<code>(puste)</code>	Token systemowy KSeF
<code>KSEF_AUTH_POLL_INTERVAL_SEC</code>	<code>1.0</code>	Interwał pollingu statusu auth
<code>KSEF_AUTH_POLL_MAX_ATTEMPTS</code>	<code>60</code>	Max prób (timeout ~60 s)
<code>OLLAMA_HOST</code>	<code>http://localhost:11434</code>	Adres serwera Ollama
<code>OLLAMA_MODEL</code>	<code>qwen2.5:7b</code>	Model SLM do kategoryzacji

Konfiguracja jest singletonem: `get_settings()` w `app/config.py` z dekoratorem `@lru_cache` — wczytuje `.env` raz przy starcie procesu.

4. Baza danych: modele, migracje, RLS

4.1. Model danych (ER)



4.2. Tabele — szczegóły kolumn

tenants

Kolumna	Typ	Opis
id	UUID PK	Identyfikator dzierżawcy (firma MŚP)
name	VARCHAR(255)	Nazwa firmy
nip	VARCHAR(10) UNIQUE	NIP — klucz biznesowy
ksef_hwm_date	TIMESTAMPTZ NULL	High Water Mark — ostatnia data synchronizacji KSeF
created_at	TIMESTAMPTZ	Data utworzenia rekordu

tenant_categories

Kolumna	Typ	Opis
id	UUID PK	
tenant_id	UUID FK → tenants	Właściciel kategorii
name	VARCHAR(128)	Nazwa kategorii kosztowej (słownik AI)
sort_order	INTEGER	Kolejność wyświetlania / promptu
is_active	BOOLEAN	Czy kategoria jest aktywna

Indeks unikalny: (tenant_id, name)

invoices

Kolumna	Źródło FA(3)	Opis
invoice_number	P_2	Numer faktury
issue_date	P_1	Data wystawienia
sale_date	P_6	Data sprzedaży (opcjonalna)
seller_nip	Podmiot1/NIP	NIP sprzedawcy
buyer_nip	Podmiot2/NIP	NIP nabywcy
ksef_number	KSeF	Numer KSeF (unikalny, opcjonalny w teście)
currency_code	—	Domyślnie PLN

invoice_lines

Kolumna	Źródło FA(3)	Opis
line_number	kolejność FaWiersz	Numer wiersza (1-based)
product_name	P_7	Nazwa towaru/usługi — jedyne pole wysyłane do AI
quantity	P_8B	Ilość (NUMERIC 18,4)
unit_price	P_9A	Cena jednostkowa netto
line_net_value	P_11	Wartość netto wiersza — podstawa agregacji statystyk
ai_category_main	AI	Główna kategoria z drzewa tenanta
ai_category_sub	AI	Podkategoria (max 3 słowa)
ai_confidence	AI	Pewność 0–100

4.3. Row-Level Security (RLS)

Izolacja multi-tenant jest wymuszana na poziomie PostgreSQL, nie tylko w kodzie aplikacji. Mechanizm opiera się na zmiennej sesyjnej `app.current_tenant`.

```
-- Ustawienie kontekstu (app/database/session.py → set_tenant_context):
SELECT set_config('app.current_tenant', '<uuid-tenanta>', true);
-- trzeci argument true = SET LOCAL (reset po COMMIT/ROLLBACK)

-- Polityka RLS (migracja 001, tabele invoices, invoice_lines, tenant_categories):
CREATE POLICY tenant_isolation_policy ON <table>
FOR ALL
USING (tenant_id = NULLIF(current_setting('app.current_tenant', true), '')::UUID)
WITH CHECK (tenant_id = NULLIF(current_setting('app.current_tenant', true), '')::UUID);
```

Migracja 003 (FORCE RLS): Właściciel tabeli (użytkownik aplikacji) nie może omijać RLS. Bez tego PostgreSQL domyślnie pozwala ownerowi widzieć wszystkie wiersze.

Dwutorowe zabezpieczenie w analityce: `StatisticsService` dodatkowo filtruje `WHERE InvoiceLine.tenant_id = self._tenant_id` — jawny filtr w SQL jako defense-in-depth.

4.4. Historia migracji Alembic

Rewizja	Plik	Zmiany
001_initial	001_initial_schema_with_qls.py	tenants, invoices, invoice_lines + ENABLE RLS + polityki
002_tenant_categories	002_tenant_categories.py	Tabela tenant_categories + RLS + seed domyślnych nazw
003_force_qls	003_force_qls.py	FORCE ROW LEVEL SECURITY na 3 tabelach tenant-scoped

5. Uwierzytelnianie i multi-tenancy

5.1. Model JWT (tryb deweloperski)

W MVP nie ma pełnego logowania użytkownika. Token JWT jest generowany skryptem `scripts/create_dev_token.py` i zawiera:

```
{
  "sub": "dev-user",
  "tenant_id": "<UUID tenanta z bazy>",
  "exp": <timestamp UTC + 24h>
}
```

5.2. Przepływ autoryzacji w żądaniu HTTP

1. Klient wysyła nagłówek `Authorization: Bearer <token>`

2. `get_current_tenant` (`dependencies/tenant.py`) dekoduje JWT kluczem `JWT_SECRET_KEY`
3. Wyciąga `tenant_id` → tworzy obiekt `TenantContext`
4. `get_rls_session` otwiera transakcję async, wywołuje `set_tenant_context`, yielduje sesję SQLAlchemy
5. Po zakończeniu handlera: COMMIT (lub ROLLBACK przy błędzie)

5.3. Middleware TenantRLSMiddleware

Plik `middleware/tenant_rls.py` — opcjonalnie parsuje JWT i ustawia `request.state.tenant_id`. **Główna ścieżka RLS to dependency injection** (`get_rls_session`), nie middleware. Middleware jest pomocniczy i nie blokuje żądań przy nieprawidłowym tokenie.

6. Warstwa HTTP API

6.1. Rejestracja routerów

```
app/main.py
├── include_router(api_router)      # prefix /api/v1
└── stats.router                   # prefix /stats
```

6.2. Pełna lista endpointów

Metoda	Ścieżka	Auth	Handler	Odpowiedź
GET	/health	Nie	main.py	<code>{"status":"ok","app":"Wirtualny CFO"}</code>
GET	/api/v1/me	JWT	main.py	<code>{"tenant_id":"...", "sub":"dev-user"}</code>
GET	/api/v1/invoice-lines	JWT+RLS	main.py	Lista max 100 wierszy (<code>InvoiceLineRead</code>)
GET	/api/v1/stats/cost-structure	JWT+RLS	stats.py	<code>CostStructureResponse</code>
GET	/api/v1/stats/trend	JWT+RLS	stats.py	<code>TrendResponse</code>
GET	/api/v1/stats/top-vendors	JWT+RLS	stats.py	<code>TopVendorsResponse</code>

6.3. Parametry query (endpointy stats)

Parametr	Typ	Domyślnie	Działanie
<code>date_from</code>	date (ISO)	null	Filtr <code>Invoice.issue_date >= date_from</code>
<code>date_to</code>	date (ISO)	null	Filtr <code>Invoice.issue_date <= date_to</code>
<code>limit</code>	int 1–10	10	Tylko top-vendors — liczba dostawców

6.4. CORS

W `main.py` dozwolone originy: `http://localhost:3000`, `http://127.0.0.1:3000` — umożliwia fetch z serwera Next.js.

Niezaimplementowane w API: upload XML, trigger ETL, synchronizacja KSeF, CRUD tenantów. Te operacje istnieją wyłącznie jako serwisy wewnętrzne lub skrypty CLI.

7. Potok ETL (XML → AI → PostgreSQL)

Centralna klasa: `InvoiceEtlPipeline` w `app/services/etl_pipeline.py`. Orkiestruje cały przepływ przetwarzania pojedynczej faktury FA(3).

7.1. Diagram sekwencji ETL

Wywołujący: skrypt CLI lub (przyszły) endpoint API

- 0 process_invoice_xml(tenant_id, xml_bytes, ksef_number?, session?)**
 - Jeśli `session` podany — używa zewnętrznej transakcji
 - Jeśli `session=None` — otwiera własną sesję + `set_tenant_context`
- 1 parse_fa3_xml(xml_content)**
 - Bezpieczny parser lxml (ochrona XXE)
 - Wynik: `Fa3ParseResult { header, lines[] }`
- 2 resolve_tenant_categories(session, tenant_id)**
 - `SELECT` z `tenant_categories` WHERE `is_active`
 - Fallback: `DEFAULT_TENANT_CATEGORIES` (6 kategorii MVP)
- 3 INSERT Invoice**
 - Nagłówek z `parsed.header` (`P_1`, `P_2`, NIP-y...)
 - `session.flush()` → uzyskanie `invoice.id`
- 4 FOR EACH line IN parsed.lines**

4a.

```
categorizer.classify_product_name(line.product_name, allowed_categories)
→ do Ollamy trafia
wyłącznie P_7
(nazwa towaru/usługi)
```

4b.

```
INSERT InvoiceLine — P_7, P_8B, P_9A, P_11 + ai_category_*
```

5

session.flush() → EtlProcessResult

- Zwraca: `tenant_id`, `invoice_id`, `lines_processed`, `categories_used`

7.2. Zarządzanie transakcją

- Gdy `session=None`: pipeline sam otwiera sesję, `begin()`, `set_tenant_context`, `commit()` lub `rollback()`
- Gdy `session` przekazany (jak w `test_etl_pipeline.py`): pipeline używa zewnętrznej transakcji — caller robi `commit()`

7.3. Wynik EtlProcessResult

Pole	Typ	Znaczenie
<code>tenant_id</code>	UUID	Tenant, dla którego zapisano fakturę
<code>invoice_id</code>	UUID	ID nowego rekordu invoices
<code>lines_processed</code>	int	Liczba przetworzonych wierszy FaWiersz
<code>categories_used</code>	list[str]	Lista kategorii użytych w prompcie AI

7.4. Przykład end-to-end (fixture testowy)

Plik: `backend/tests/fixtures/sample_fa3_invoice.xml`

Wiersz	P_7	P_11	Typowa kategoria AI
1	Elektrody spawalnicze 3.2mm	255.00	Materiały i Surowce
2	Olej napędowy ON	245.00	Paliwa i Transport

Nagłówek: data P_1 = 2026-01-15, sprzedawca NIP 5265877635. Suma netto = 500.00 PLN.

7.5. Skrypt test_etl_pipeline.py — krok po kroku

1. `ensure_test_tenant()` — szuka NIP `9998887776`, tworzy tenant + 6 kategorii jeśli brak
2. `set_tenant_context(session, tenant_id)` — aktywuje RLS
3. `pipeline.process_invoice_xml(...)` — pełny ETL na fixture XML
4. `session.commit()` — trwały zapis
5. Ponowne odczytanie `InvoiceLine` i wypisanie klasyfikacji AI w konsoli
6. Podpowiedź: `create_dev_token.py --nip 9998887776`

8. Parser FA(3) XML

Plik: `app/services/xml/fa3_parser.py`

8.1. Bezpieczeństwo parsowania

```
etree.XMLParser(  
    resolve_entities=False,    # ochrona przed XXE  
    no_network=True,          # brak dostępu sieciowego z parsera  
    dtd_validation=False,  
    load_dtd=False,  
    huge_tree=False,          # ochrona przed Billion Laughs  
)
```

8.2. Namespace i XPath

- Domyślny namespace FA(3): `http://crd.gov.pl/wzor/2025/06/25/13775/`
- Wykrywanie dynamiczne z atrybutu `xmlns` korzenia dokumentu
- Wiersze: XPath `//tns:Fa/tns:FaWiersz` + fallback bez namespace
- Nagłówek: P_1, P_2, P_6 z sekcji Fa; NIP z Podmiot1/Podmiot2

8.3. Modele wynikowe (`xml/models.py`)

Model	Pola
Fa3InvoiceHeader	invoice_number, issue_date, sale_date, seller_nip, buyer_nip, currency_code
Fa3InvoiceLine	line_number, product_name, quantity, unit_price, line_net_value
Fa3ParseResult	namespace, header, lines: list[Fa3InvoiceLine]

8.4. Wyjątki

`Fa3XmlParserError` — nieprawidłowy XML, brak wierszy, brak P_7, błędne Decimal/date. Opakowywany przez ETL jako `EtlPipelineError`.

9. Moduł kategoryzacji AI (Ollama)

9.1. Klasa ProductCategorizer

Plik: `app/services/ai/categorizer.py`

Przeływ `classify_product_name(product_name, allowed_categories)`

1. `validate_allowed_categories()` — lista niepusta, bez duplikatów
2. `_assert_product_name_isolation(product_name)` — walidacja P_7:
 - niepuste, max 512 znaków
 - brak NIP (regex 10 cyfr)
 - brak kwot pieniężnych (wzorzec z 2 miejscami po przecinku lub separatorem tysięcy)
 - brak adresu e-mail

3. `build_category_schema(categories)` — dynamiczny Pydantic z `Literal[kat1, kat2, ...]`
4. `build_category_json_schema()` — JSON Schema dla parametru `format Ollama`
5. `build_system_prompt(categories)` — prompt systemowy z listą dozwolonych kategorii
6. Wywołanie `AsyncClient.chat()` :
 - `model: OLLAMA_MODEL` (domyślnie `qwen2.5:7b`)
 - `temperature: 0.0`
 - `num_predict: 128`
 - `format: JSON Schema (structured output)`
7. `_sanitize_model_payload()` — czyszczenie artefaktów JSON z odpowiedzi SLM
8. `schema_cls.model_validate(payload)` — walidacja Pydantic

9.2. Domyślne kategorie (`ai/schemas.py`)

1. Materiały i Surowce
2. Opakowania
3. Paliwa i Transport
4. Koszty Biurowe i IT
5. Usługi Zewnętrzne
6. Inne Koszty Operacyjne

9.3. Schemat odpowiedzi AI

```
{
  "kategoria_glowna": "<Literal z listy tenanta>",
  "kategoria_podrzeczna": "<max 3 słowa po polsku>",
  "pewnosc_klasyfikacji": 0-100
}
```

9.4. Wyjątki

Klasa	Przyczyna
<code>AllInputIsolationError</code>	Naruszenie reguł izolacji P_7
<code>AI CategorizationError</code>	Błąd Ollama, nieprawidłowy JSON, walidacja Pydantic

10. Integracja KSeF API 2.0

Moduł KSeF implementuje **wyłącznie uwierzytelnianie**. Pobieranie faktur (ZIP, HWM, eksport) nie jest jeszcze zaimplementowane — pole `ksef_hwm_date` w tabeli `tenants` jest przygotowane na przyszłość.

10.1. Komponenty

Plik	Klasa / funkcja	Rola
ksef/client.py	KsefClient	httpx async — wywołania REST
ksef/auth.py	KsefAuthService	Orkiestracja pełnego flow auth
ksef/crypto.py	encrypt_ksef_token	RSA-OAEP SHA-256: token timestampMs
ksef/models.py	Pydantic	Modele request/response API KSeF

10.2. Sekwencja uwierzytelniania tokenem KSeF

1

POST /auth/challenge

Pobranie jednorazowego challenge i znacznika czasu

→ `challenge`, `timestampMs`

2

GET /security/public-key-certificates

Pobranie certyfikatów publicznych MF do szyfrowania tokena

→ wybór certyfikatu (`public_key_id`)

3

RSA-OAEP SHA-256

Szyfrowanie ciągu: `ksef_token + "|" + timestampMs`

→ `encrypted_token` (base64), `public_key_id`

4

POST /auth/ksef-token

Body: `challenge`, `contextIdentifier: {type:"Nip", value: NIP}`, `encrypted_token`, `public_key_id`

→ `reference_number`, `authentication_token`

5

POLL GET /auth/{referenceNumber}

Nagłówek: `Authorization: Bearer <authentication_token>`

→ kod **100** = w toku | kod **200** = sukces | max 60 prób × 1 s

6

POST /auth/token/redeem

Nagłówek: `Authorization: Bearer <authentication_token>` (jednorazowo)

→ `access_token`, `refresh_token` (+ `valid_until`)

Uwaga implementacyjna: NIP kontekstu biznesowego idzie do `contextIdentifier` w `/auth/ksef-token`, nie do `/auth/challenge`.

Test: `scripts/test_ksef_auth.py` — wymaga `KSEF_NIP` i `KSEF_TOKEN` w `.env`.

11. Analityka opisowa (StatisticsService)

Plik: app/services/analytics/statistics.py

Klasa przyjmuje (session, tenant_id) i wykonuje agregacje SQL przez SQLAlchemy 2.

11.1. get_cost_structure() — wykres kołowy

```
SELECT COALESCE(ai_category_main, 'Niesklasyfikowane') AS category,
       SUM(line_net_value) AS total_net
FROM invoice_lines
JOIN invoices ON invoice_lines.invoice_id = invoices.id
WHERE invoice_lines.tenant_id = :tenant_id
      [AND issue_date BETWEEN date_from AND date_to]
GROUP BY category
ORDER BY total_net DESC
```

Odpowiedź: CostStructureResponse { items[], total_net, date_from, date_to }

11.2. get_spending_trend() — trend miesięczny

```
SELECT to_char(date_trunc('month', issue_date), 'YYYY-MM') AS month,
       SUM(line_net_value) AS total_net
FROM invoice_lines JOIN invoices ...
WHERE tenant_id = :tenant_id
GROUP BY date_trunc('month', issue_date)
ORDER BY month ASC
```

11.3. get_top_vendors() — ranking dostawców

```
SELECT seller_nip, SUM(line_net_value) AS total_net
FROM invoice_lines JOIN invoices ...
WHERE tenant_id = :tenant_id
GROUP BY seller_nip
ORDER BY total_net DESC
LIMIT :limit -- max 10
```

Ranking (rank) jest nadawany w Pythonie (enumerate od 1).

11.4. Przykładowe JSON dla fixture (1 faktura)

```
GET /stats/trend
→ { "items": [{ "month": "2026-01", "total_net": "500.00" }], "total_net": "500.00" }

GET /stats/cost-structure
→ { "items": [
  { "category": "Materiały i Surowce", "total_net": "255.00" },
  { "category": "Paliwa i Transport", "total_net": "245.00" }
], "total_net": "500.00" }

GET /stats/top-vendors?limit=5
→ { "items": [{ "seller_nip": "5265877635", "total_net": "500.00", "rank": 1 } ] }
```

12. Konsumpcja API przez frontend (bez warstwy wizualnej)

Frontend (Next.js 16, App Router) pełni rolę cienkiego klienta. Interesujące z punktu widzenia architektury:

12.1. Konfiguracja (frontend/.env.local)

Zmienna	Domyślnie	Rola
NEXT_PUBLIC_API_BASE_URL	http://127.0.0.1:8000/api/v1	Bazowy URL API
NEXT_PUBLIC_DEV_TOKEN	—	JWT Bearer wstawiany do każdego fetch

12.2. Klient HTTP (frontend/lib/api.ts)

- `apiFetch(path, params?)` — GET z nagłówkiem Authorization, cache: no-store
- `toNumber()` — konwersja Decimal string → number (PostgreSQL zwraca stringi w JSON)
- `formatPln()` — formatowanie PLN dla tooltipów

12.3. Komponenty pobierające dane (useEffect przy mount)

Komponent	Endpoint	Typ odpowiedzi
TrendChart.tsx	/stats/trend	TrendResponse
CostStructureChart.tsx	/stats/cost-structure	CostStructureResponse
TopVendorsList.tsx	/stats/top-vendors?limit=5	TopVendorsResponse

Strona `app/page.tsx` montuje te 3 komponenty. Każdy niezależnie wykonuje fetch — brak wspólnego state managera (React Query/SWR). Błędy API wyświetlane jako tekst czerwony.

12.4. Warunek poprawnego działania dashboardu

1. Backend uvicorn na porcie 8000

- 2. JWT w .env.local z tenant_id identycznym jak dane w bazie
- 3. JWT_SECRET_KEY backendu = klucz użyty przy generowaniu tokena
- 4. Co najmniej 1 faktura w invoice_lines dla tego tenanta

13. Skrypty deweloperskie

Skrypt	Wymagania	Co robi
create_dev_token.py	PostgreSQL (opcjonalnie dla --nip)	Generuje JWT 24h. Args: UUID lub --nip 9998887776
test_etl_pipeline.py	PostgreSQL + Ollama + migracje	Pełny ETL na fixture XML, tenant testowy NIP 9998887776
test_parse_fa3_xml.py	Brak (offline)	Parsuje XML, wypisuje JSON nagłówka i wierszy
test_ai_categorization.py	Ollama	Testuje 3 przykładowe nazwy produktów bez bazy
test_ksef_auth.py	KSEF_NIP, KSEF_TOKEN w .env	Pełny flow uwierzytelniania KSeF, wypisuje tokeny

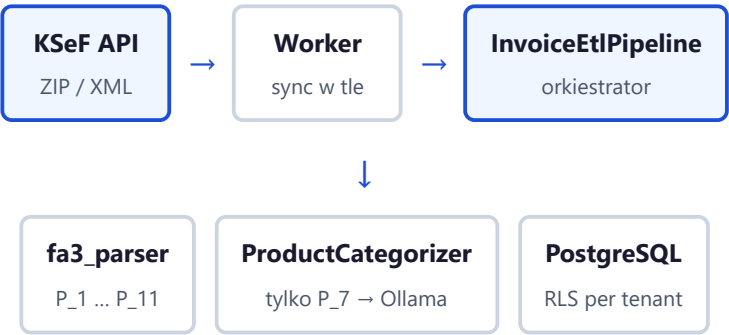
14. Testy automatyczne

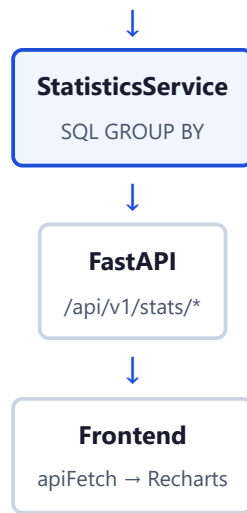
Uruchomienie: cd backend && .venv\Scripts\pytest tests/ -v

Plik	Zakres	Baza
test_fa3_parser.py	Parsowanie fixture, XXE, brak wierszy	—
test_ai_categorizer.py	Izolacja P_7, dynamiczny schemat, prompt	—
test_dynamic_schema.py	Literal validation, injection w prompt	—
test_etl_pipeline.py	Pełny ETL z mockiem AI	SQLite async (aiosqlite)
test_statistics.py	3 endpointy stats + filtry dat	SQLite async

15. Diagramy przepływu danych

15.1. Od XML do wykresu (pełna ścieżka produkcyjna docelowa)





15.2. Izolacja tenanta w żądaniu HTTP

1. Przeglądarka: `fetch` + nagłówek `Authorization: Bearer <JWT>`
2. FastAPI `get_current_tenant` — dekodowanie JWT, wyciągnięcie `tenant_id`
3. `get_rls_session` — `BEGIN` + `set_config('app.current_tenant', uuid, true)`
4. `StatisticsService` — `SELECT` z `invoice_lines` JOIN `invoices`
5. PostgreSQL RLS — polityka filtruje wiersze: `tenant_id = app.current_tenant`
6. Odpowiedź JSON do klienta
7. `COMMIT` — zamknięcie transakcji, reset `SET LOCAL`

16. Mapowanie FA(3) → baza danych

Element XML FA(3)	XPath / lokalizacja	Tabela.kolumna
Numer faktury	Fa/P_2	invoices.invoice_number
Data wystawienia	Fa/P_1	invoices.issue_date
Data sprzedaży	Fa/P_6	invoices.sale_date
NIP sprzedawcy	Podmiot1/*/NIP	invoices.seller_nip
NIP nabywcy	Podmiot2/*/NIP	invoices.buyer_nip
Numer KSeF	(metadane KSeF)	invoices.ksef_number
Nazwa towaru	Fa/FaWiersz/P_7	invoice_lines.product_name → AI
Ilość	Fa/FaWiersz/P_8B	invoice_lines.quantity
Cena jedn. netto	Fa/FaWiersz/P_9A	invoice_lines.unit_price
Wartość netto	Fa/FaWiersz/P_11	invoice_lines.line_net_value
—	Ollama output	invoice_lines.ai_category_main/sub/confidence

17. Luki implementacyjne i roadmap

Obszar	Status	Uwagi
Auth produkcyjne (OAuth/login)	Brak	Tylko dev JWT
REST upload XML / trigger ETL	Brak	Tylko skrypt CLI
KSeF pobieranie faktur (ZIP, HWM)	Brak	Tylko auth client
Worker tła (Celery/ARQ)	Brak	ETL synchroniczny w skrypcie
InflationEngine / anomalie	Brak	Świadomie odłożone w planie MVP
Refresh token KSeF rotation	Brak	redeem zwraca tokeny, brak przechowywania
RLS na tenants	Brak (zamierzone)	Lookup po NIP bez kontekstu tenanta